

Discrete-Time Systems

Example

Determine the response of the following systems to the input signal

$$x(n) = \begin{cases} |n|, & -3 \leq n \leq 3 \\ 0, & \text{otherwise} \end{cases}$$

(a) $y(n) = x(n)$

(b) $y(n) = x(n - 1)$

(c) $y(n) = x(n + 1)$

(d) $y(n) = \frac{1}{3}[x(n + 1) + x(n) + x(n - 1)]$

(e) $y(n) = \max\{x(n + 1), x(n), x(n - 1)\}$

(f) $y(n) = \sum_{k=-\infty}^n x(k) = x(n) + x(n - 1) + x(n - 2) + \dots$

Example

- (a) In this case the output is exactly the same as the input signal. Such a system is known as the *identity* system.
- (b) This system simply delays the input by one sample. Thus its output is given by

$$x(n) = \{\dots, 0, 3, 2, 1, 0, 1, 2, 3, 0, \dots\}$$

↑

- (c) In this case the system “advances” the input one sample into the future. For example, the value of the output at time $n = 0$ is $y(0) = x(1)$. The response of this system to the given input is

$$x(n) = \{\dots, 0, 3, 2, 1, 0, 1, 2, 3, 0, \dots\}$$

↑

- (d) The output of this system at any time is the mean value of the present, the immediate past, and the immediate future samples. For example, the output at time $n = 0$ is

$$y(0) = \frac{1}{3}[x(-1) + x(0) + x(1)] = \frac{1}{3}[1 + 0 + 1] = \frac{2}{3}$$

Repeating this computation for every value of n , we obtain the output signal

$$y(n) = \{\dots, 0, 1, \frac{5}{3}, 2, 1, \frac{2}{3}, 1, 2, \frac{5}{3}, 1, 0, \dots\}$$

↑

Example

- (e) This system selects as its output at time n the maximum value of the three input samples $x(n-1)$, $x(n)$, and $x(n+1)$. Thus the response of this system to the input signal $x(n)$ is

$$y(n) = \{0, 3, 3, 3, 2, 1, 2, 3, 3, 3, 0, \dots\}$$

↑

- (f) This system is basically an *accumulator* that computes the running sum of all the past input values up to present time. The response of this system to the given input is

$$y(n) = \{\dots, 0, 3, 5, 6, 6, 7, 9, 12, 0, \dots\}$$

↑

Building Block 1: The Adder

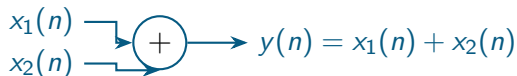
Definition

An **adder** sums two (or more) signal sequences to produce an output sequence:

$$y(n) = x_1(n) + x_2(n)$$

Key Property

The addition operation is *memoryless* — no storage of past values is required.



Graphical representation of an adder

Building Block 2: Constant Multiplier

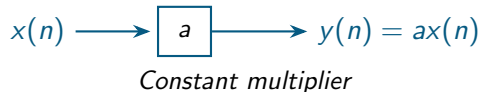
Definition

A **constant multiplier** scales the input by a fixed constant a :

$$y(n) = ax(n)$$

Key Property

This operation is also *memoryless*.



Building Block 3: Signal Multiplier

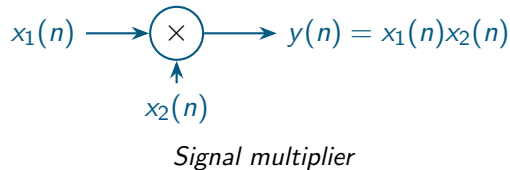
Definition

A **signal multiplier** forms the element-wise product of two sequences:

$$y(n) = x_1(n) \cdot x_2(n)$$

Key Property

Like the adder, this operation is *memoryless*.



Building Block 4: Unit Delay Element

Definition

The **unit delay** delays the input by one sample:

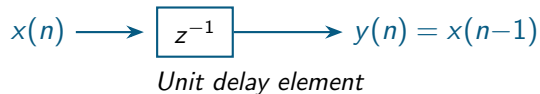
$$y(n) = x(n - 1)$$

The sample $x(n - 1]$ is stored in memory at time $n - 1$ and recalled at time n .

Key Property

This block **requires memory**.

Notation: z^{-1} (related to the z -transform).



Building Block 5: Unit Advance Element

Definition

The **unit advance** shifts the input forward by one sample:

$$y(n) = x(n + 1)$$

Notation: z (the advance operator).



Figure 2.17 — Unit advance element

Physical Limitation

A unit advance is **physically impossible** in real time because it requires knowledge of a future sample.

It is, however, feasible in non-real-time (stored) processing.

Summary of Building Blocks

Block	Input–Output relation	Memory?	Symbol
Adder	$y(n) = x_1(n) + x_2(n)$	No	$\oplus$
Constant Multiplier	$y(n) = a x(n)$	No	$[a]$
Signal Multiplier	$y(n) = x_1(n) \cdot x_2(n)$	No	$\otimes$
Unit Delay	$y(n) = x(n - 1)$	Yes	$[z^{-1}]$
Unit Advance	$y(n) = x(n + 1)$	Yes	$[z]$

Example 2.2.3 — Problem Statement

Given System

Sketch the block diagram of the discrete-time system described by:

$$y(n) = \frac{1}{4} y(n-1) + \frac{1}{2} x(n) + \frac{1}{2} x(n-1) \quad (2.2.5)$$

where $x(n)$ is the input and $y(n)$ is the output.

Decomposition of the output:

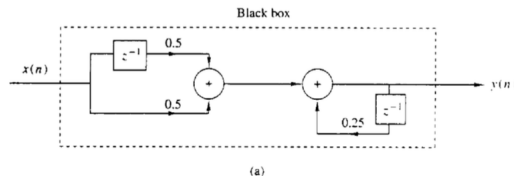
- Multiply current input $x(n)$ by 0.5
- Multiply previous input $x(n-1)$ by 0.5
- Add the two products
- Add the previous output $y(n-1)$ multiplied by 0.25

Example 2.2.3 — Realisation (a)

Direct Form (Equation 2.2.5)

Two separate delay elements — one on the **input** path, one on the **output** (feedback) path.

- z^{-1} on input: produces $x(n-1)$, scaled by 0.5
- z^{-1} on output: produces $y(n-1)$, scaled by 0.25
- Two adders combine all three weighted signals



Example 2.2.3 — Realisation (b)

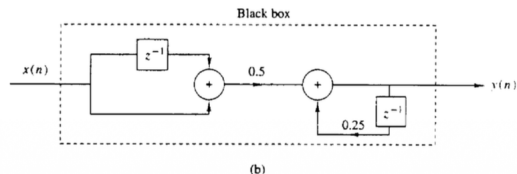
Simplified Form (Equation 2.2.6)

Rearranging:

$$y(n] = \frac{1}{4}y(n-1) + \frac{1}{2}[x(n) + x(n-1)] \quad (2.2.6)$$

- A single z^{-1} element shared for $x(n-1)$
- One adder sums $x(n) + x(n-1)$, then multiplied by 0.5
- Feedback path multiplied by 0.25

Both forms are **equivalent** input-output descriptions.



Static vs Dynamic Systems

Static (Memoryless)

Output at n depends **only** on $x(n)$:

$$y(n) = \mathcal{T}[x(n), n]$$

No delay elements. No stored values.

Examples:

- $y(n) = ax(n)$
- $y(n) = nx(n) + bx^3(n)$

Dynamic (Has Memory)

Output depends on **past** (or future) samples.
Contains z^{-1} delay element(s).

- **Finite memory** N : uses $x(n - N) \dots x(n)$
- **Infinite memory**: $N = \infty$

Examples:

- $y(n) = x(n) + 3x(n - 1)$ finite, $N = 1$
- $y(n) = \sum_{k=0}^{\infty} x(n - k)$ infinite

Rule

$N = 0 \Rightarrow$ static $0 < N < \infty \Rightarrow$ finite memory $N = \infty \Rightarrow$ infinite memory

Time-Invariant vs Time-Variant — The Core Idea

The core question: Does the system behave the same way *no matter when* you apply the input?

Time-Invariant (TI)

The system's “rule” does **not change** with time. If you delay the input by k samples, the output simply delays by the same k — nothing else changes.

$$x(n) \xrightarrow{\mathcal{T}} y(n) \Rightarrow x(n-k) \xrightarrow{\mathcal{T}} y(n-k)$$

Analogy: A fixed resistor — apply a voltage now or 5 seconds later, you always get the same scaled current. The resistor never changes.

Time-Variant (TV)

The behaviour **depends on when** the input is applied. Delaying the input does *not* simply delay the output; you get a different result.

Typical giveaway: n appears as an explicit multiplier of the signal, e.g. $y(n) = nx(n)$.

Analogy: A microphone whose gain is physically increased every second — the same word spoken at $t = 1$ s vs $t = 10$ s gives very different outputs.

Time-Invariant — The 3-Step Test Explained

① Path A: “Delay then Process”

Replace $x(n)$ with $x(n - k)$ in the system equation.

Compute $y(n, k) = \mathcal{T}[x(n - k)]$.

② Path B: “Process then Delay”

Take the original output $y(n) = \mathcal{T}[x(n)]$ and substitute $n \rightarrow n - k$ to get $y(n - k)$.

③ Compare

- $y(n, k) = y(n - k)$ for all $k \Rightarrow$ **Time Invariant**
- $y(n, k) \neq y(n - k)$ for even one $k \Rightarrow$ **Time Variant**

Why this test works

A TI system is like a machine whose internal gears never rust or change. It does not matter whether you wait k seconds before feeding the input (Path A) or wait k seconds after reading the output (Path B) — the result is identical. A TV system has gears that wear over time, so the two paths give different results.

Example 2.2.4 — Testing Four Systems

System	$y(n, k)$ $\mathcal{T}[x(n-k)]$	$=$	$y(n-k)$	Result
(a) $y(n) = x(n) - x(n-1)$	$x(n-k) - x(n-k-1)$		$x(n-k) - x(n-k-1)$	TI
(b) $y(n) = nx(n)$	$n \cdot x(n-k)$		$(n-k)x(n-k)$	TV
(c) $y(n) = x(-n)$	$x(-(n-k))$ $x(-n+k)$	$=$	$x(-n+k)$ different in- dex origin	TV
(d) $y(n) = x(n) \cos \omega_0 n$	$x(n-k) \cos \omega_0 n$		$x(n-k) \cos \omega_0 (n-k)$	TV

- **(a) Differentiator:** structure fixed — **time invariant**
- **(b) Time multiplier** $nx(n)$: gain changes with n — **time variant**
- **(c) Folder** $x(-n)$: time reversal causes mismatch — **time variant**
- **(d) Modulator** $x(n) \cos \omega_0 n$: carrier not shifted — **time variant**

Linear Systems — What is Superposition?

The core question: If you know how a system responds to each input *individually*, can you predict its response to any *combination* of those inputs? For a linear system: **yes**.

Formal Definition (2.2.26)

A relaxed system $\mathcal{T}$ is **linear** if and only if:

$$\mathcal{T}[a_1x_1(n) + a_2x_2(n)] = a_1\mathcal{T}[x_1(n)] + a_2\mathcal{T}[x_2(n)]$$

for *all* inputs $x_1(n)$, $x_2(n)$ and *all* constants a_1 , a_2 .

This single equation encodes **two independent properties** that must *both* hold:

Linear Systems — What is Superposition?

Property 1: Scaling (Homogeneity)

$$\mathcal{T}[a x(n)] = a \mathcal{T}[x(n)] = a y(n)$$

If you scale the input by a , the output scales by the *same* factor a . Double the input $\Rightarrow$ double the output. Halve the input $\Rightarrow$ halve the output. The system never “amplifies differently” for different signal sizes.

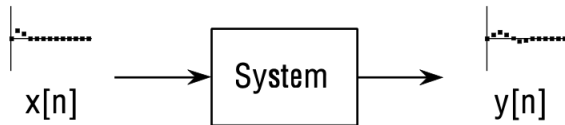
Property 2: Additivity

$$\mathcal{T}[x_1(n) + x_2(n)] = y_1(n) + y_2(n)$$

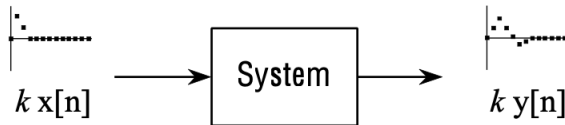
The response to a *sum* of two signals equals the *sum* of the individual responses. You can analyse each input separately and simply add the results — the inputs do not “interact” inside the system.

Linear Systems — What is Superposition?

IF

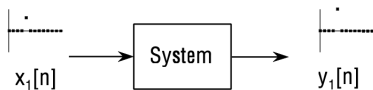


THEN

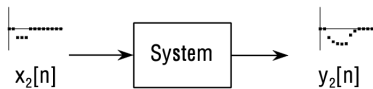


Linear Systems — What is Superposition?

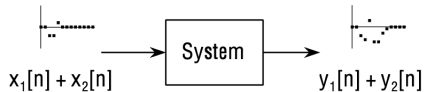
IF



AND IF



THEN



Superposition — Intuition and Extended Form

Intuition: Think of a system as a black box. Superposition says you can “decompose” any complicated input into simple pieces, find the output for each piece, then add those outputs together. This is what makes linear systems tractable to analyse.

Analogy: An amplifier. If you play note A alone and get output y_A , and note B alone gives y_B , then playing both notes together gives $y_A + y_B$ (for a *linear* amplifier). A *nonlinear* amplifier would produce distortion — extra frequencies not in either original note.

General (Extended) Superposition (2.2.29)

$$x(n) = \sum_{k=1}^{M-1} a_k x_k(n) \xrightarrow{\mathcal{T}} y(n) = \sum_{k=1}^{M-1} a_k y_k(n), \quad y_k(n) = \mathcal{T}[x_k(n)]$$

Any weighted combination of M inputs produces the same weighted combination of their individual outputs.

Nonlinear systems violate this

$y(n) = x^2(n)$: if $x(n) = x_1(n) + x_2(n)$, then $y(n) = x_1^2 + 2x_1x_2 + x_2^2 \neq y_1(n) + y_2(n)$. The cross-term $2x_1x_2$ breaks additivity.

Linear vs Nonlinear — Examples

System	Equation	Reason	Linear?
Amplifier	$y(n) = ax(n)$	Pure scaling	Yes
Accumulator	$y(n) = \sum_{k=-\infty}^n x(k)$	Satisfies superposition	Yes
Time mult.	$y(n) = nx(n)$	Satisfies superposition	Yes
Square law	$y(n) = x^2(n)$	$\mathcal{T}[2x] \neq 2\mathcal{T}[x]$	No
DC offset	$y(n) = x(n) + 5$	Additivity fails	No

Common trap

$y(n) = x(n) + 5$ **looks** linear but is **not** — the constant offset 5 breaks the additivity property.